

# Algorithm Analysis Fundamentals

Time complexity analysis of iterative and recursive algorithms; verify Big-O,  $\Omega$ ,  $\Theta$  for sample programs

Dr. Poongavanam N

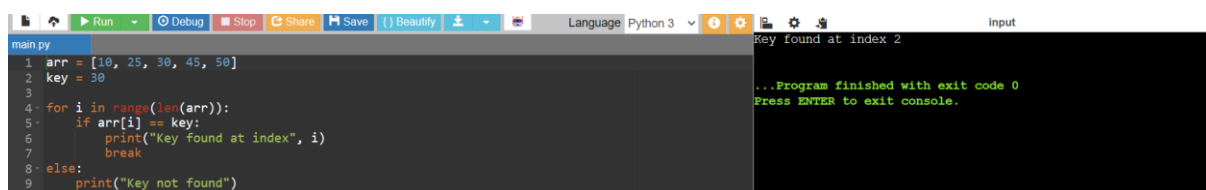
NAME: Y. Jithendra Reddy

Reg.no: 192525303

DAA Topic 01 Experiments

## Experiment 1: Linear Search (Iterative)

- Objective: To analyse time complexity of linear search.
- Problem Statement: Write a Python program to search for a key in an array using linear search.
- Sample Input: [10, 25, 30, 45, 50], Key = 30
- Sample Output: Key found at index 2
- Complexity:  $\Omega$ :  $O(1)$ ,  $O$ :  $O(n)$ ,  $\Theta$ :  $\Theta(n)$
- Explanation: Demonstrates sequential scanning and inefficiency for large inputs.



```
main.py
1 arr = [10, 25, 30, 45, 50]
2 key = 30
3
4 for i in range(len(arr)):
5     if arr[i] == key:
6         print("Key found at index", i)
7         break
8 else:
9     print("Key not found")
```

Input

Key found at index 2

...Program finished with exit code 0  
Press ENTER to exit console.

## Experiment 2: Binary Search (Recursive)

- Objective: To compare recursive binary search complexity.
- Problem Statement: Implement recursive binary search in Python.
- Sample Input: [5, 10, 15, 20, 25], Key = 20

- Sample Output: Key found at index 3
- Complexity:  $\Omega: O(1)$ ,  $O: O(\log_{f_0} n)$ ,  $\Theta: \Theta(\log_{f_0} n)$
- Explanation: Shows divide-and-conquer and logarithmic efficiency.

```

1 def binary_search(arr, key, low, high):
2     if low > high:
3         return -1
4
5     mid = (low + high) // 2
6
7     if arr[mid] == key:
8         return mid
9     elif key < arr[mid]:
10        return binary_search(arr, key, low, mid - 1)
11    else:
12        return binary_search(arr, key, mid + 1, high)
13
14
15 arr = [5, 10, 15, 20, 25]
16 key = 20
17
18 result = binary_search(arr, key, 0, len(arr) - 1)
19
20 if result != -1:
21     print("Key found at index", result)
22 else:
23     print("Key not found")

```

Key found at index 3

...Program finished with exit code 0  
Press ENTER to exit console.

### Experiment 3: Factorial (Iterative vs Recursive)

- Objective: To compare iterative and recursive factorial computation.
- Problem Statement: Write programs to compute factorial using both methods.
- Sample Input:  $n = 5$
- Sample Output: 120
- Complexity: Both  $O(n)$
- Explanation: Highlights recursion depth vs iterative loops.

```

1 def iterative_factorial(n):
2     result = 1
3
4     for i in range(1, n + 1):
5         result = result * i
6
7     return result
8
9
10 def recursive_factorial(n):
11     if n == 0 or n == 1:
12         return 1
13
14     return n * recursive_factorial(n - 1)
15
16
17 n = 5
18
19 print("Iterative factorial:", iterative_factorial(n))
20 print("Recursive factorial:", recursive_factorial(n))

```

Iterative factorial: 120  
Recursive factorial: 120

...Program finished with exit code 0  
Press ENTER to exit console.

### Experiment 4: Fibonacci Series

- Objective: To analyze recursive vs iterative Fibonacci generation.
- Problem Statement: Generate first  $n$  Fibonacci numbers.

- Sample Input:  $n = 6$
- Sample Output:  $[0, 1, 1, 2, 3, 5]$
- Complexity: Iterative  $O(n)$ , Recursive naïve  $O(2n)$ , Recursive with memorization  $O(n)$
- Explanation: Illustrates exponential recursion vs efficient iteration.

```

1 def fibonacci(n):
2     series = []
3     a, b = 0, 1
4
5     for i in range(n):
6         series.append(a)
7         a, b = b, a + b
8
9     return series
10
11
12 n = 6
13 print(fibonacci(n))
14
15 def fibonacci_number(n):
16     if n <= 1:
17         return n
18
19     return fibonacci_number(n - 1) + fibonacci_number(n - 2)
20
21
22 n = 6
23
24 for i in range(n):
25     print(fibonacci_number(i), end=" ")

```

Output: [0, 1, 1, 2, 3, 5]  
 ...Program finished with exit code 0  
 Press ENTER to exit console.

## Experiment 5: Matrix Multiplication

- Objective: To analyse nested loop complexity.
- Problem Statement: Multiply two matrices using loops.
- Sample Input:  $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ ,  $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$
- Sample Output:  $\begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$
- Complexity:  $O(n^3)$
- Explanation: Shows cubic growth due to nested loops.

```

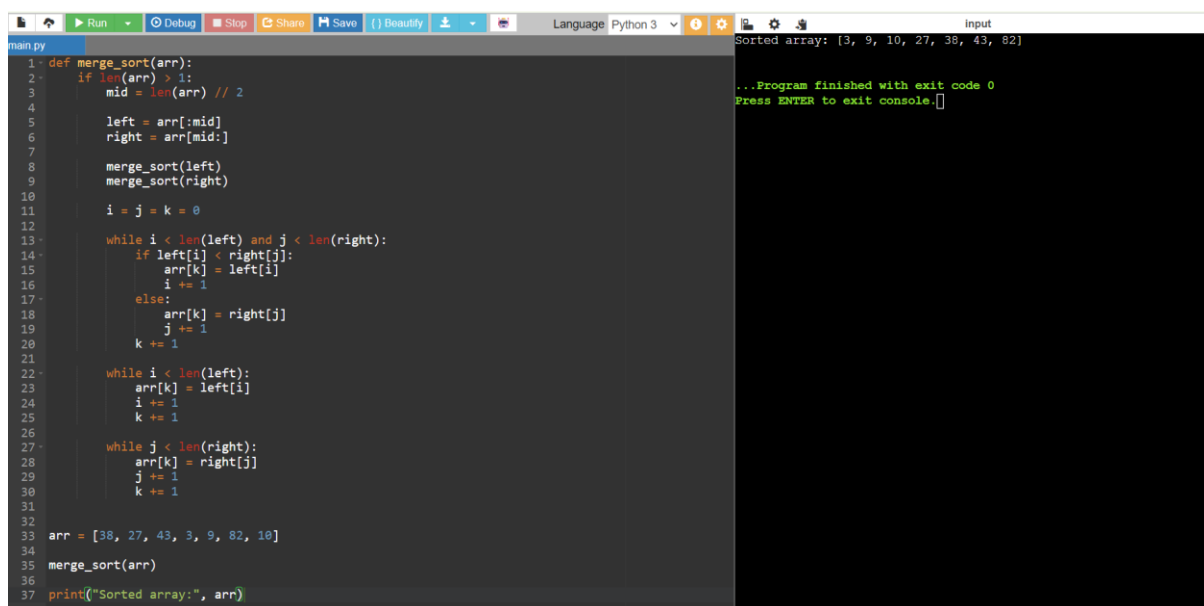
1 A = [
2     [1, 2],
3     [3, 4]
4 ]
5
6 B = [
7     [5, 6],
8     [7, 8]
9 ]
10
11 result = [
12     [0, 0],
13     [0, 0]
14 ]
15
16 for i in range(len(A)):
17     for j in range(len(B[0])):
18         for k in range(len(B)):
19             result[i][j] += A[i][k] * B[k][j]
20
21 print("Result:")
22
23 for row in result:
24     print(row)

```

Result:  
 [19, 22]  
 [43, 50]  
 ...Program finished with exit code 0  
 Press ENTER to exit console.

## Experiment 6: Merge Sort

- Objective: To analyse divide-and-conquer sorting.
- Problem Statement: Implement merge sort.
- Sample Input: [38, 27, 43, 3, 9, 82, 10]
- Sample Output: [3, 9, 10, 27, 38, 43, 82]
- Complexity: Always  $O(n \log n)$



```
1 def merge_sort(arr):
2     if len(arr) > 1:
3         mid = len(arr) // 2
4         left = arr[:mid]
5         right = arr[mid:]
6         merge_sort(left)
7         merge_sort(right)
8         i = j = k = 0
9         while i < len(left) and j < len(right):
10             if left[i] < right[j]:
11                 arr[k] = left[i]
12                 i += 1
13             else:
14                 arr[k] = right[j]
15                 j += 1
16             k += 1
17         while i < len(left):
18             arr[k] = left[i]
19             i += 1
20             k += 1
21         while j < len(right):
22             arr[k] = right[j]
23             j += 1
24             k += 1
25     arr = [38, 27, 43, 3, 9, 82, 10]
26     merge_sort(arr)
27     print("Sorted array:", arr)
```

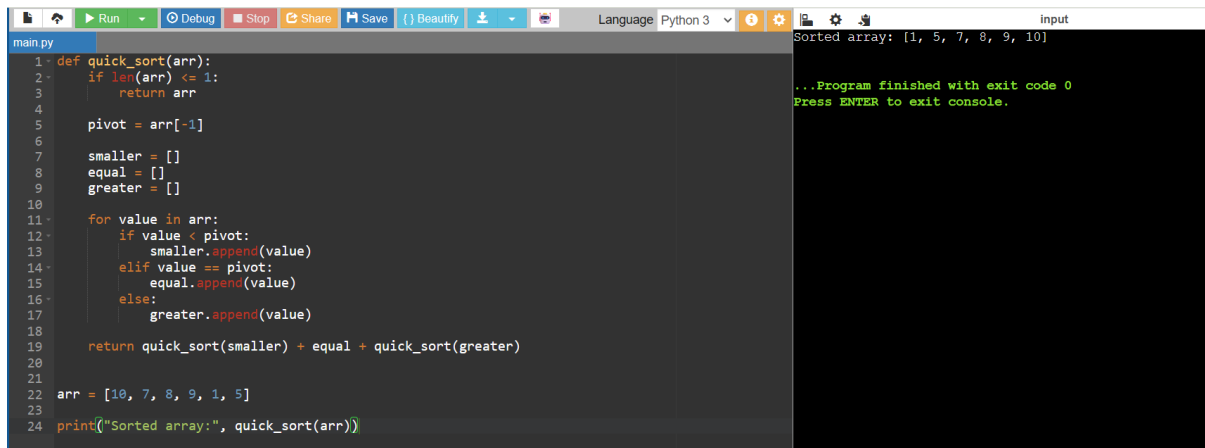
Sorted array: [3, 9, 10, 27, 38, 43, 82]

...Program finished with exit code 0  
Press ENTER to exit console.

- Explanation: Demonstrates guaranteed efficiency of divide-and-conquer.

## Experiment 7: Quick Sort

- Objective: To analyse recursive quick sort.
- Problem Statement: Implement quick sort.
- Sample Input: [10, 7, 8, 9, 1, 5]
- Sample Output: [1, 5, 7, 8, 9, 10]
- Complexity:  $\Omega: O(n \log n)$ ,  $O: O(n^2)$ ,  $\Theta: \Theta(n \log n)$
- Explanation: Shows how pivot choice affects performance.



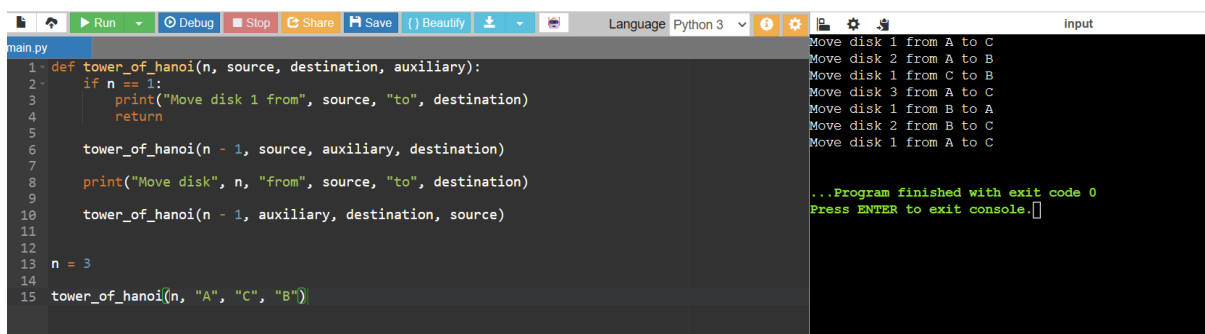
```
main.py
1 def quick_sort(arr):
2     if len(arr) <= 1:
3         return arr
4
5     pivot = arr[-1]
6
7     smaller = []
8     equal = []
9     greater = []
10
11     for value in arr:
12         if value < pivot:
13             smaller.append(value)
14         elif value == pivot:
15             equal.append(value)
16         else:
17             greater.append(value)
18
19     return quick_sort(smaller) + equal + quick_sort(greater)
20
21
22 arr = [10, 7, 8, 9, 1, 5]
23
24 print("Sorted array:", quick_sort(arr))
```

Sorted array: [1, 5, 7, 8, 9, 10]

...Program finished with exit code 0  
Press ENTER to exit console.

## Experiment 8: Tower of Hanoi

- Objective: To analyze exponential recursion.
- Problem Statement: Solve Tower of Hanoi for n disks.
- Sample Input: n = 3
- Sample Output: Sequence of moves.
- Complexity:  $O(2^n)$
- Explanation: Illustrates exponential growth in recursive calls.



```
main.py
1 def tower_of_hanoi(n, source, destination, auxiliary):
2     if n == 1:
3         print("Move disk 1 from", source, "to", destination)
4         return
5
6     tower_of_hanoi(n - 1, source, auxiliary, destination)
7
8     print("Move disk", n, "from", source, "to", destination)
9
10    tower_of_hanoi(n - 1, auxiliary, destination, source)
11
12
13 n = 3
14
15 tower_of_hanoi(n, "A", "C", "B")
```

Move disk 1 from A to C  
Move disk 2 from A to B  
Move disk 1 from C to B  
Move disk 3 from A to C  
Move disk 1 from B to A  
Move disk 2 from B to C  
Move disk 1 from A to C

...Program finished with exit code 0  
Press ENTER to exit console.

## Experiment 9: GCD (Euclidean Algorithm)

- Objective: To analyze recursive GCD.
- Problem Statement: Implement Euclidean algorithm.
- Sample Input: a=48, b=18

- Sample Output: 6
- Complexity:  $O(\log_{f_0} n)$
- Explanation: Shows efficient recursion with logarithmic complexity.

The screenshot shows a Python IDE with a file named 'main.py'. The code defines a recursive function 'gcd(a, b)' that returns 'a' if 'b' is 0, otherwise it returns 'gcd(b, a % b)'. Below the function, variables 'a' and 'b' are set to 48 and 18 respectively, and the function is called with 'print("GCD:", gcd(a, b))'. The output console on the right shows 'GCD: 6' and a message indicating the program finished with exit code 0.

```

1 def gcd(a, b):
2     if b == 0:
3         return a
4
5     return gcd(b, a % b)
6
7
8 a = 48
9 b = 18
10
11 print("GCD:", gcd(a, b))

```

Output: GCD: 6

...Program finished with exit code 0  
Press ENTER to exit console.

## Experiment 10: Insertion Sort

- Objective: To analyze iterative sorting.
- Problem Statement: Implement insertion sort.
- Sample Input: [12, 11, 13, 5, 6]
- Sample Output: [5, 6, 11, 12, 13]
- Complexity:  $\Omega$ :  $O(n)$ ,  $O$ :  $O(n^2)$ ,  $\Theta$ :  $\Theta(n^2)$
- Explanation: Demonstrates input sensitivity (sorted vs unsorted).

The screenshot shows a Python IDE with a file named 'main.py'. The code defines an 'insertion\_sort(arr)' function that sorts an array in place. Below the function, an array 'arr' is initialized with [12, 11, 13, 5, 6], the function is called, and the sorted array is printed. The output console on the right shows 'Sorted array: [5, 6, 11, 12, 13]' and a message indicating the program finished with exit code 0.

```

1 def insertion_sort(arr):
2     for i in range(1, len(arr)):
3         key = arr[i]
4         j = i - 1
5
6         while j >= 0 and arr[j] > key:
7             arr[j + 1] = arr[j]
8             j -= 1
9
10        arr[j + 1] = key
11
12
13 arr = [12, 11, 13, 5, 6]
14 insertion_sort(arr)
15 print("Sorted array:", arr)

```

Sorted array: [5, 6, 11, 12, 13]

...Program finished with exit code 0  
Press ENTER to exit console.

## Experiment 11: Heap Sort

- Objective: To analyze heap-based sorting.
- Problem Statement: Implement heap sort.
- Sample Input: [4, 10, 3, 5, 1]
- Sample Output: [1, 3, 4, 5, 10]
- Complexity: Always  $O(n \log_{f_0} n)$

- Explanation: Consistent performance using heap structure.

```

1 def heapify(arr, n, i):
2     largest = i
3     left = 2 * i + 1
4     right = 2 * i + 2
5
6     if left < n and arr[left] > arr[largest]:
7         largest = left
8
9     if right < n and arr[right] > arr[largest]:
10        largest = right
11
12    if largest != i:
13        arr[i], arr[largest] = arr[largest], arr[i]
14
15        heapify(arr, n, largest)
16
17
18 def heap_sort(arr):
19     n = len(arr)
20
21     for i in range(n // 2 - 1, -1, -1):
22         heapify(arr, n, i)
23
24     for i in range(n - 1, 0, -1):
25         arr[0], arr[i] = arr[i], arr[0]
26
27         heapify(arr, i, 0)
28
29
30 arr = [4, 10, 3, 5, 1]
31
32 heap_sort(arr)
33
34 print("Sorted array:", arr)

```

Sorted array: [1, 3, 4, 5, 10]

...Program finished with exit code 0  
Press ENTER to exit console.

## Experiment 12: Counting Inversions

- Objective: To analyze modified merge sort.
- Problem Statement: Count inversions in an array.
- Sample Input: [2, 4, 1, 3, 5]
- Sample Output: 3
- Complexity:  $O(n \log^2 n)$
- Explanation: Shows divide-and-conquer applied beyond sorting.

```

1 def count_inversions(arr):
2     count = 0
3
4     for i in range(len(arr)):
5         for j in range(i + 1, len(arr)):
6             if arr[i] > arr[j]:
7                 count += 1
8
9     return count
10
11
12 arr = [2, 4, 1, 3, 5]
13
14 print("Number of inversions:", count_inversions(arr))

```

Number of inversions: 3

...Program finished with exit code 0  
Press ENTER to exit console.

## Experiment 13: Maximum Subarray Sum

- Objective: To compare Kadane's vs divide-and-conquer.
- Problem Statement: Find maximum subarray sum.
- Sample Input: [-2, -3, 4, -1, -2, 1, 5, -3]
- Sample Output: 7
- Complexity: Kadane's  $O(n)$ , Divide & Conquer  $O(n \log n)$
- Explanation: Highlights efficiency differences between approaches.

```

main.py
1 def maximum_subarray_sum(arr):
2     current_sum = arr[0]
3     maximum_sum = arr[0]
4
5     for i in range(1, len(arr)):
6         current_sum = max(arr[i], current_sum + arr[i])
7         maximum_sum = max(maximum_sum, current_sum)
8
9     return maximum_sum
10
11
12 arr = [-2, -3, 4, -1, -2, 1, 5, -3]
13
14 print("Maximum subarray sum:", maximum_subarray_sum(arr))

```

Maximum subarray sum: 7

...Program finished with  
Press ENTER to exit console

## Experiment 14: Exponentiation

- Objective: To compare iterative vs recursive fast power.
- Problem Statement: Compute  $x^n$ .
- Sample Input:  $x=2, n=10$
- Sample Output: 1024
- Complexity: Iterative  $O(n)$ , Recursive fast power  $O(\log n)$
- Explanation: Shows how recursion reduces complexity.



```
main.py
1 def iterative_power(x, n):
2     result = 1
3
4     for i in range(n):
5         result = result * x
6
7     return result
8
9
10 x = 2
11 n = 10
12
13 print("Result:", iterative_power(x, n))
14 def fast_power(x, n):
15     if n == 0:
16         return 1
17
18     half = fast_power(x, n // 2)
19
20     if n % 2 == 0:
21         return half * half
22     else:
23         return x * half * half
24
25
26 x = 2
27 n = 10
28
29 print("Result:", fast_power(x, n))
```

Result: 1024  
Result: 1024  
...Program finished with exit code 0  
Press ENTER to exit console.

## Experiment 15: Closest Pair of Points

- Objective: To analyze brute force vs divide-and-conquer.
- Problem Statement: Find closest pair of points in 2D.
- Sample Input: [(1,2), (4,5), (7,8), (3,1)]
- Sample Output: Closest pair (1,2)-(3,1), Distance =  $\sqrt{5}$
- Complexity: Brute force  $O(n^2)$ , Optimized  $O(n \log n)$
- Explanation: Demonstrates quadratic complexity and motivates optimization.

```
main.py
1 import math
2
3
4 def closest_pair(points):
5     minimum_distance = float("inf")
6     pair = None
7
8     for i in range(len(points)):
9         for j in range(i + 1, len(points)):
10             x1, y1 = points[i]
11             x2, y2 = points[j]
12
13             distance = math.sqrt(
14                 (x2 - x1) ** 2 + (y2 - y1) ** 2
15             )
16
17             if distance < minimum_distance:
18                 minimum_distance = distance
19                 pair = (points[i], points[j])
20
21     return pair, minimum_distance
22
23
24 points = [(1, 2), (4, 5), (7, 8), (3, 1)]
25
26 pair, distance = closest_pair(points)
27
28 print("Closest pair:", pair)
29 print("Distance:", distance)
```

Closest pair: ((1, 2), (3, 1))  
Distance: 2.23606797749979  
...Program finished with exit code 0  
Press ENTER to exit console.